Descriere Arhitecturală
StudySync AI

Cuprins

1 Introducere **3**

2 Sinteză a Produsului Rezultat **3**
2.1 Descrierea Generală . . . 3
2.2 Comparație cu Produsul Propus Inițial . . . 3
2.2.1 Funcționalități Implementate . . . 3
2.2.2 Îmbunătățiri și Modificări . . . 4

3 Descriere folosind Diagrame C4 **4**
3.1 System Context Diagram . . . 4
3.2 Container Diagram . . . 5
3.3 Component Diagram . . . 6
3.3.1 Controller Layer (REST Endpoints) . . . 7
3.3.2 Service Layer (Business Logic) . . . 8
3.3.3 Repository Layer (Data Access) . . . 8

4 Cerințe Non-Funcționale și Soluții Arhitecturale **8**
4.1 Performanță . . . 8
4.1.1 Cerință . . . 8
4.1.2 Soluții Arhitecturale . . . 8
4.2 Scalabilitate . . . 9
4.2.1 Cerință . . . 9
4.2.2 Soluții Arhitecturale . . . 9
4.3 Mentenabilitate . . . 10
4.3.1 Cerință . . . 10
4.3.2 Soluții Arhitecturale . . . 10
4.4 Securitate . . . 10
4.4.1 Cerință . . . 10
4.4.2 Soluții Arhitecturale . . . 11
4.5 Disponibilitate . . . 11
4.5.1 Cerință . . . 11
4.5.2 Soluții Arhitecturale . . . 11
4.6 Portabilitate . . . 12
4.6.1 Cerință . . . 12
4.6.2 Soluții Arhitecturale . . . 12
4.7 Usabilitate . . . 12

4.7.1 Cerință 12
4.7.2 Soluții Arhitecturale 12

5 Stack Tehnologic **13**
5.1 Frontend 13
5.2 Backend (API) 13
5.3 ML Server 13
5.4 Database 13
5.5 DevOps și Deployment 14

6 Concluzii **14**

1 Introducere

Acest document prezintă arhitectura sistemului **StudySync AI**, o platformă inteligentă de management al documentelor educaționale, care utilizează tehnologii de inteligență artificială pentru a facilita procesul de învățare colaborativă. Documentul urmărește să descrie deciziile arhitecturale luate în cadrul proiectului și să ofere o viziune completă asupra structurii sistemului.

2 Sinteză a Produsului Rezultat

2.1 Descrierea Generală

StudySync AI este o aplicație web care permite studenților să:

- Creeze și gestioneze *Study Spaces* (spații de studiu) pentru diferite cursuri sau proiecte
- Încarce documente în formate diverse (PDF, TXT, MD, DOCX)
- Efectueze căutări inteligente bazate pe AI folosind tehnologia RAG (Retrieval-Augmented Generation)
- Colaboreze cu alți studenți prin comentarii contextuale (similar cu Notion)
- Comunice între ei prin chat (mesaje între utilizatori)
- Gestioneze calendare și evenimente legate de studiu

2.2 Comparație cu Produsul Propus Inițial

2.2.1 Funcționalități Implementate

Produsul final a reușit să implementeze majoritatea funcționalităților propuse la livrabilul intermediar:

1. **Autentificare și Gestionare Utilizatori:** Sistem complet de înregistrare, login și resetare parolă
2. **Study Spaces:** Creare și gestionare de spații de studiu colaborative cu invitații prin link-uri
3. **Gestionare Documente:** Upload, vizualizare, și ștergere documente cu suport pentru multiple formate
4. **AI Passage Finder:** Motor de căutare RAG care identifică pasaje relevante din documentele indexate
5. **Chat între utilizatori:** Sistem de mesagerie pentru comunicare directă între membri
6. **Chat între utilizatori:** Sistem de mesagerie pentru comunicare directă între membri
7. **Calendar:** Funcționalitate de gestionare a evenimentelor și termenelor

2.2.2 Îmbunătățiri și Modificări

Pe parcursul dezvoltării, au fost aduse următoarele îmbunătățiri față de planul inițial:

- **Arhitectură Microservicii:** Separarea completă a serverului ML de backend pentru scalabilitate
- **Performanță Optimizată:** Indexare în sub 30 de secunde pentru documente de 50 de pagini
- **Concurență:** Suport pentru minimum 10 cereri simultane de căutare
- **Parsare Avansată:** Sistem unificat de parsare a documentelor cu OCR pentru PDF-uri scanate
- **Vector Search:** Implementare cu Sentence Transformers și căutare cosinus similarity

3 Descriere folosind Diagrame C4

Modelul C4 oferă o metodologie structurată de documentare a arhitecturii software prin patru nivele de abstractizare. Pentru StudySync AI, am creat trei tipuri de diagrame care descriu sistemul de la nivel înalt până la detalii de implementare.

3.1 System Context Diagram

System Context Diagram prezintă imaginea de ansamblu, arătând cum sistemul StudySync AI interacționează cu utilizatorii și sistemele externe.

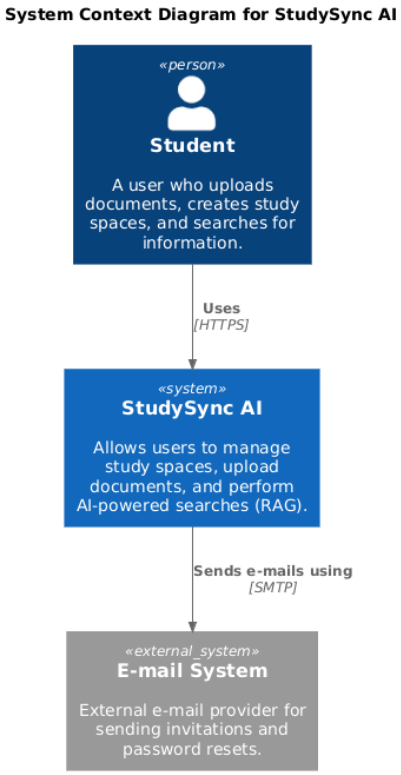


Figura 1: System Context Diagram pentru StudySync AI

Componente principale:

- **Student:** Utilizatorul principal care încarcă documente, creează spații de studiu și efectuează căutări
- **StudySync AI System:** Sistemul principal care gestionează toate funcționalitățile
- **E-mail System:** Sistem extern pentru trimiterea invitațiilor și resetarea parolelor

3.2 Container Diagram

Diagrama de containere dezvăluie arhitectura tehnică la nivel înalt, prezentând aplicațiile și serviciile care compun sistemul.

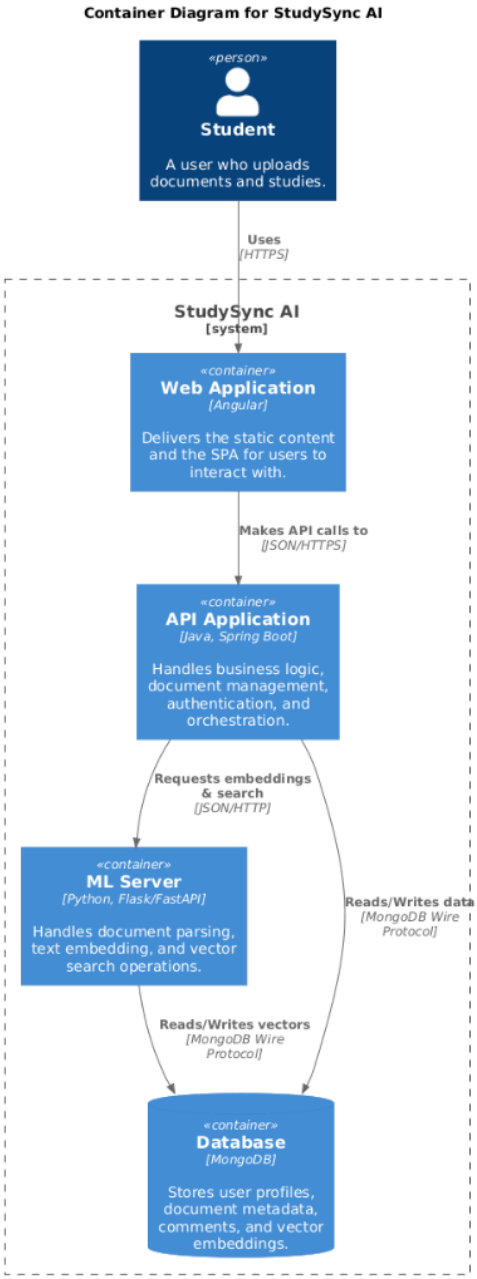


Figura 2: Container Diagram pentru StudySync AI

Containere identificate:

1. **Web Application (Angular):**
   - Single Page Application (SPA) care servește interfața utilizator
   - Comunicare prin HTTPS cu API-ul backend
   - Framework: Angular cu TypeScript
2. **API Application (Spring Boot):**
   - Gestionează logica de business
   - Autentificare și autorizare utilizatori
   - Orchestrare între frontend, ML Server și baza de date
   - Framework: Java cu Spring Boot
3. **ML Server (Python):**
   - Parsare documente (PDF, DOCX, TXT, MD)
   - Generare embeddings vectoriali folosind Sentence Transformers
   - Căutare semantică (vector search)
   - Framework: Python cu Flask/FastAPI
4. **Database (MongoDB):**
   - Stocarea profilului utilizatorilor
   - Metadata documente și spații de studiu
   - Vectori embeddings pentru căutare
   - Comentarii și interacțiuni colaborative

3.3 Component Diagram

Diagrama de componente detaliază structura internă a API Application (Spring Boot), prezentând responsabilitățile fiecărei componente.

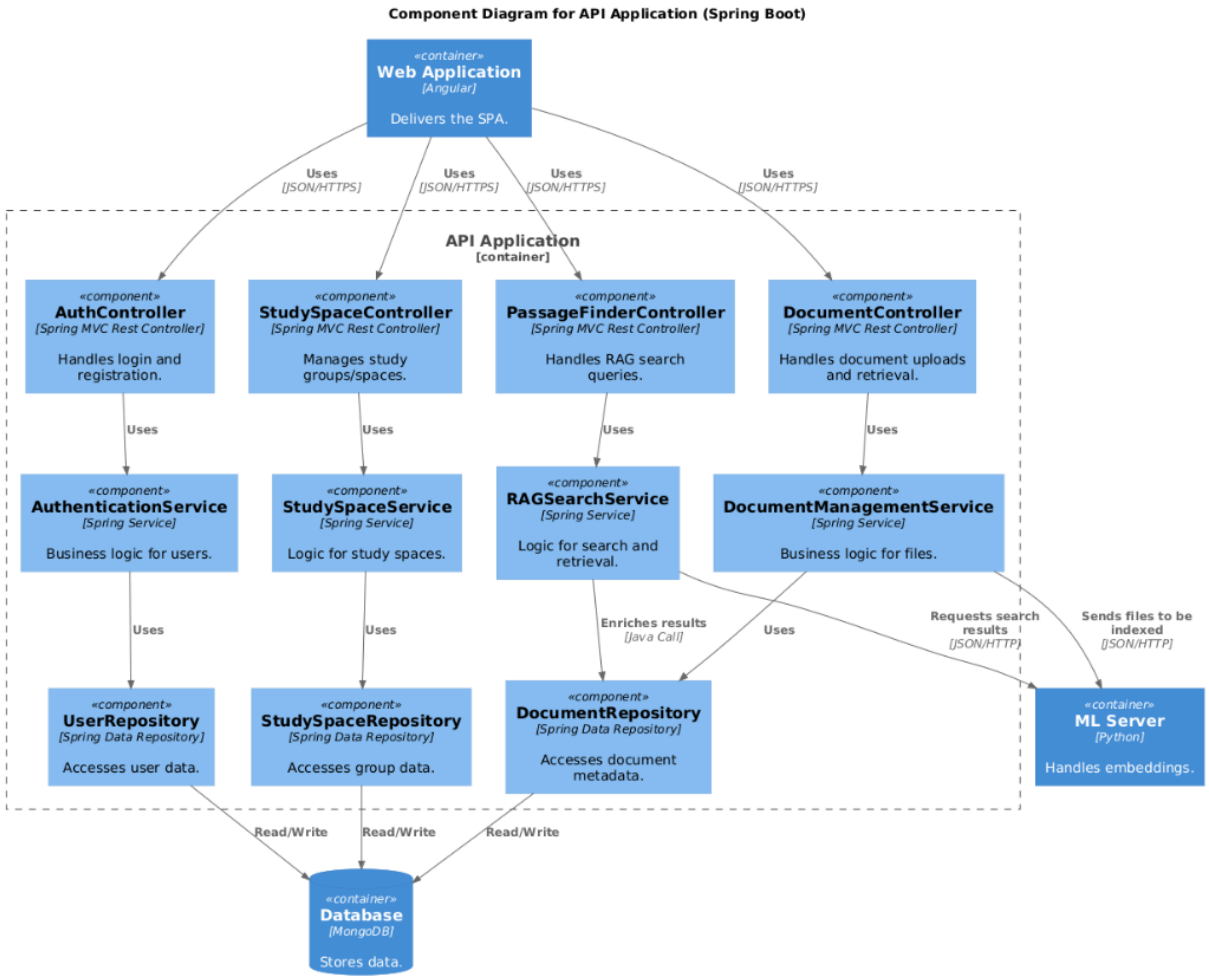


Figura 3: Component Diagram pentru API Application (Spring Boot)

Arhitectură în straturi:

3.3.1 Controller Layer (REST Endpoints)

- **AuthController:** Gestionează autentificarea (login, register, reset password)
- **StudySpaceController:** CRUD operații pentru spații de studiu, invitații membri
- **DocumentController:** Upload, retrieval, ștergere documente
- **PassageFinderController:** Procesează query-uri de căutare RAG
- **ChatController:** Endpoints pentru chatbot-ul AI general
- **CommentController:** Gestionează comentariile pe documente
- **CalendarController:** Gestionează evenimente și deadline-uri

3.3.2 Service Layer (Business Logic)

- **AuthenticationService:** Logica de autentificare și validare utilizatori
- **StudySpaceService:** Logica de management a grupurilor de studiu
- **DocumentManagementService:** Procesare și organizare fișiere
- **RAGSearchService:** Orchestrare căutare și retrieval, comunicare cu ML Server
- **FileConversionService:** Conversie formate documente
- **ChatService:** Integrare cu API-uri externe pentru chatbot
- **CommentService:** Logica pentru comentarii și mențiuni
- **CalendarService:** Logica pentru evenimente

3.3.3 Repository Layer (Data Access)

- **UserRepository:** Acces la datele utilizatorilor
- **StudySpaceRepository:** Acces la datele spațiilor de studiu
- **DocumentRepository:** Acces la metadata documentelor
- Utilizează Spring Data MongoDB pentru abstractizare

4 Cerințe Non-Funcționale și Soluții Arhitecturale

4.1 Performanță

4.1.1 Cerință

- Indexare document de 50 de pagini: sub 30 de secunde
- Suport pentru un minim de 10 cereri de căutare simultane fără timeout
- Timp de răspuns pentru căutări: sub 2 secunde

4.1.2 Soluții Arhitecturale

1. Separarea ML Server:

- ML Server rulează ca proces independent
- Permite scalare orizontală independentă
- Nu blochează thread-urile din Spring Boot

2. Procesare Asincronă:

- Indexarea documentelor se face asincron
- Utilizare de job queues pentru task-uri intensive

- Notificări către frontend la finalizare

3. Optimizarea Algoritmilor de Căutare:

- Utilizare Sentence Transformers pre-antrenați
- Căutare vectorială optimizată cu cosine similarity
- Indexare chunk-uri de text (nu documente întregi)

4. Caching:

- Cache pentru embeddings frecvent utilizați
- Cache rezultate căutări populare
- Reducerea apelurilor către ML Server

4.2 Scalabilitate

4.2.1 Cerință

- Suport pentru creșterea numărului de utilizatori
- Gestionare volume mari de documente
- Posibilitatea de extindere a capacității de procesare

4.2.2 Soluții Arhitecturale

1. Arhitectură Microservicii:

- API Application și ML Server sunt servicii independente
- Fiecare poate fi scalat independent
- Deployment separat și versioning independent

2. Bază de Date NoSQL (MongoDB):

- Schema flexibilă pentru evoluție ușoară
- Suport nativ pentru scalare orizontală (sharding)
- Performanță bună pentru operații de citire intensivă

3. Stateless API:

- API-ul Spring Boot nu păstrează stare sesiune
- Permite load balancing simplu
- Multiple instanțe pot rula în paralel

4.3 Mentenabilitate

4.3.1 Cerință

- Cod ușor de întreținut și extins
- Separare clară a responsabilităților
- Testabilitate ridicată

4.3.2 Soluții Arhitecturale

1. Arhitectură în Straturi (Layered Architecture):

- *Controller Layer:* Gestionează cererile HTTP
- *Service Layer:* Conține logica de business
- *Repository Layer:* Acces la date
- Separare clară, dependențe unidirecționale

2. Dependency Injection:

- Utilizare Spring IoC Container
- Loose coupling între componente
- Ușor de înlocuit implementări pentru testing

3. Design Patterns:

- *Repository Pattern:* Pentru acces la date
- *Service Pattern:* Pentru logică de business
- *DTO Pattern:* Pentru transfer de date între straturi

4.4 Securitate

4.4.1 Cerință

- Protecție date utilizatori
- Acces controlat la resurse
- Comunicare securizată

4.4.2 Soluții Arhitecturale

1. Autentificare și Autorizare:

- JWT (JSON Web Tokens) pentru sesiuni
- Password hashing cu BCrypt
- Role-based access control pentru Study Spaces

2. Comunicare HTTPS:

- Toate cererile între frontend și backend prin HTTPS
- Certificat SSL/TLS pentru producție

3. Validare Input:

- Validare la nivel de controller cu Bean Validation
- Sanitizare input pentru prevenirea XSS și injection attacks
- Limitare dimensiune fișiere uploadate

4. Izolare Servicii:

- ML Server nu este expus direct către internet
- Comunicare doar prin API Application
- Firewall rules și network segmentation

4.5 Disponibilitate

4.5.1 Cerință

- Sistem disponibil 24/7
- Minimizarea timpului de nefuncționare
- Recovery rapid în caz de erori

4.5.2 Soluții Arhitecturale

1. Health Checks:

- Spring Boot Actuator pentru monitoring
- Endpoints de health check pentru fiecare serviciu
- Automatic restart la detectarea problemelor

2. Error Handling:

- Global exception handlers
- Logging complet cu nivele de severitate

- Circuit breaker pattern pentru apeluri externe

3. Database Redundancy:

- MongoDB replica sets pentru high availability
- Backup automat periodic
- Point-in-time recovery capabilities

4.6 Portabilitate

4.6.1 Cerință

- Deployment ușor pe diferite platforme
- Independență de infrastructură

4.6.2 Soluții Arhitecturale

1. Containerizare:

- Docker containers pentru fiecare serviciu
- Docker Compose pentru orchestrare locală
- Deployment consistent pe orice platformă

2. Configuration Management:

- Externalizare configurații (fișiere .env)
- Spring Profiles pentru diferite environmente
- Separare configurații development/production

4.7 Usabilitate

4.7.1 Cerință

- Interfață intuitivă
- Timp de învățare minim
- Feedback clar pentru utilizatori

4.7.2 Soluții Arhitecturale

1. Single Page Application (SPA):

- Utilizare Angular pentru experiență fluidă
- Navigare rapidă fără reload-uri de pagină
- State management centralizat

2. API Design Consistent:

- RESTful API conventions
- Mesaje de eroare descriptive
- Documentație API cu Swagger/OpenAPI

3. Responsive Design:

- Suport pentru multiple dispozitive
- Layout adaptiv bazat pe dimensiunea ecranului

5 Stack Tehnologic

5.1 Frontend

- **Framework:** Angular
- **Limbaj:** TypeScript
- **Package Manager:** npm
- **Build Tool:** Angular CLI

5.2 Backend (API)

- **Framework:** Spring Boot 3.x
- **Limbaj:** Java 17+
- **Build Tool:** Maven
- **Security:** Spring Security cu JWT

5.3 ML Server

- **Limbaj:** Python 3.9+
- **Framework:** Flask/FastAPI
- **ML Libraries:**
  - Sentence Transformers (embeddings)
  - PyPDF2, python-docx (parsare documente)
  - scikit-learn (vector search)

5.4 Database

- **Bază de Date:** MongoDB 6.x
- **Driver:** Spring Data MongoDB

5.5 DevOps și Deployment

- **Containerizare:** Docker
- **Orchestrare Locală:** Docker Compose
- **Version Control:** Git, GitHub

6 Concluzii

Arhitectura StudySync AI a fost concepută pentru a îndeplini cerințele funcționale și non-funcționale ale unui sistem modern de management educațional. Prin utilizarea unei arhitecturi bazate pe microservicii, separarea clară a responsabilităților și alegerea unui stack tehnologic robust, sistemul este pregătit pentru:

- **Scalare:** Atât verticală cât și orizontală
- **Mentenanță:** Cod modular și ușor de extins
- **Performanță:** Procesare rapidă și eficientă a documentelor
- **Securitate:** Protecție completă a datelor utilizatorilor
- **Evoluție:** Posibilitatea de a adăuga noi funcționalități fără refactorizări majore

Diagrama C4 oferă o viziune clară și structurată a arhitecturii, facilitând înțelegerea sistemului atât pentru echipa de dezvoltare cât și pentru stakeholderii proiectului.